

DATA SCIENCE LEARNING NOTES

DAY 1 • PYTHON FOUNDATIONS

Student: Muhammad Ahmad

Goal: Data Scientist

Track: Zero to Job Ready Data Scientist

Day 1 Revision Guide

A complete revision document covering the topics studied on Day 1. Use this PDF to revise concepts, review code, and practice before starting Day 2.

1. Variables

A variable stores a value under a name. The value can be reused later.

```
name = "Muhammad"
age = 22
cgpa = 3.23

print(name)
print(age)
print(cgpa)
```

2. Data Types

Common Python data types used in this course are String for text, Integer for whole numbers, Float for decimal numbers, and Boolean for True or False values.

```
name = "Muhammad"
age = 22
cgpa = 3.23
is_student = True

print(type(name))
print(type(age))
print(type(cgpa))
print(type(is_student))
```

3. Input and Output

input() receives user input. print() displays output. input() returns text, so convert numeric input when needed.

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
cgpa = float(input("Enter your CGPA: "))

print("Student:", name)
print("Age:", age)
print("CGPA:", cgpa)
```

4. Basic Operators

Python can perform addition, subtraction, multiplication, division, power, and remainder operations.

```
a = 10
b = 5

print(a + b)
print(a - b)
print(a * b)
```

```
print(a / b)
print(2 ** 3)
print(10 % 3)
```

5. Type Conversion

Convert data between types with `int()`, `float()`, and `str()`. This is especially important when working with input data.

```
age = "22"
age = int(age)

cgpa = "3.23"
cgpa = float(cgpa)

marks = 80
marks_text = str(marks)

print(age, type(age))
print(cgpa, type(cgpa))
print(marks_text, type(marks_text))
```

6. String Operations

Strings represent text. Important operations include length, uppercase, lowercase, trimming spaces, searching, replacing, combining, indexing, and slicing.

```
name = "Muhammad Ahmad"

print(len(name))
print(name.upper())
print(name.lower())
print(name.strip())
print("Ahmad" in name)
print(name.replace("Ahmad", "Khokhar"))
```

7. String Concatenation and f-strings

Concatenation combines strings with `+`. f-strings insert variables directly into text.

```
first_name = "Muhammad"
last_name = "Ahmad"
full_name = first_name + " " + last_name
print(full_name)

cgpa = 3.23
print(f"My name is {full_name} and my CGPA is {cgpa}")
```

8. String Indexing and Slicing

Python uses zero-based indexing. The first character is at index 0. Slicing uses `start:end`, where the end position is excluded.

```
name = "Ahmad"

print(name[0])
print(name[-1])
print(name[0:3])
```

9. Lists

A list stores multiple values. Lists are ordered and mutable, so values can be changed, added, removed, and sorted.

```
marks = [80, 75, 90, 85, 70]
```

```

print(marks[0])
print(marks[-1])
print(len(marks))

marks[1] = 88
marks.append(95)
marks.insert(1, 85)
marks.remove(70)
marks.sort()

print(marks)

```

10. Useful List Functions

For numeric lists, `sum()`, `max()`, `min()`, and `len()` are useful for quick analysis.

```

marks = [80, 75, 90, 85, 70]

total = sum(marks)
average = total / len(marks)

print("Total:", total)
print("Average:", average)
print("Highest:", max(marks))
print("Lowest:", min(marks))

```

11. Tuples

A tuple is an ordered collection whose values cannot be changed after creation. Use tuples for fixed data.

```

student = ("Ahmad", 22, 3.23)

print(student[0])
print(student[2])

```

12. Sets

A set stores unique values. Duplicate values are automatically removed. Sets are useful when you need unique categories or values.

```

grades = {"A", "B", "A", "C", "B"}
print(grades)

courses = ["Python", "SQL", "Python", "Statistics", "SQL"]
unique_courses = set(courses)
print(unique_courses)

```

13. Dictionaries

A dictionary stores information as key-value pairs. This is one of the most useful Python structures for structured data.

```

student = {
    "name": "Ahmad",
    "age": 22,
    "cgpa": 3.23
}

print(student["name"])
print(student["cgpa"])

student["cgpa"] = 3.35
student["semester"] = 7
print(student)

```

14. Lists of Dictionaries

Multiple records can be stored as a list of dictionaries. This structure is a useful bridge toward tabular data and Pandas.

```
students = [  
    {"name": "Ahmad", "cgpa": 3.23},  
    {"name": "Ali", "cgpa": 3.51},  
    {"name": "Usman", "cgpa": 3.78}  
]  
  
print(students[0]["name"])  
print(students[0]["cgpa"])
```

DAY 1 QUICK REVISION

Concept	Remember
Variable	Stores a value under a name.
String	Text, for example "Python".
Integer	Whole number, for example 22.
Float	Decimal number, for example 3.23.
Boolean	True or False.
input()	Receives user input as text.
int()	Converts a value to integer.
float()	Converts a value to decimal number.
str()	Converts a value to string.
List	Ordered and changeable collection.
Tuple	Ordered and unchangeable collection.
Set	Collection of unique values.
Dictionary	Key-value data structure.

DAY 1 PRACTICE CHECKLIST

1. Create variables for your name, age, semester, and CGPA.
2. Take two numbers from the user and calculate all basic operations.
3. Take three subject marks and calculate total and average.
4. Use string methods to clean and format a course name.
5. Create a list of six marks and calculate total, average, highest, and lowest.
6. Create a dictionary for one student and update the CGPA.
7. Create a list of three student dictionaries and print the third student's name and CGPA.

DAY 1 MINI PROJECT

Build a Student Marks Analyzer. Ask the user for the student name and marks in three subjects. Store the data in variables or a dictionary. Calculate total and average. Display the student's name, total, average, highest mark, and lowest mark.

Suggested structure:

```
student = {"name": name, "marks": [python, statistics, sql]}
```

Then use `sum()`, `len()`, `max()`, and `min()` to calculate the results.

Learning sequence

Day 1 completed: Python foundations. Day 2: if, elif, else, comparison operators, logical operators, nested conditions, and a practical grading program.

Prepared for Muhammad Ahmad • Data Science Learning Program